TruthLens: Comprehensive Technical Dossier & Viva Voce Master Guide

4th Year Capstone Project in Artificial Intelligence | Complete System Architecture & Defense Notes

Project Title: TruthLens AI Verification Suite	**Target Base LLM:** Qwen 2.5 3B Instruct (QLoRA)	**Hardware Used:** NVIDIA RTX 5050 Laptop GPU (8GB)
Deployment: Gradio 6.27 Reactive Web App	**Group Allocation:** 4 Specialized Engineering Tracks	**Status:** Phases 1–4 Fully Integrated & Tested (100%)

1. Executive Project Overview & Group Allocation Strategy

Project Motivation: Contemporary journalism, academic publishing, and online information consumption suffer from two intertwined crises: *the rampant spread of unverified/hallucinated claims* and *the inability of legacy plagiarism checkers to distinguish genuine intellectual theft from standard topical/scientific terminology overlap*. Conventional LLM fact-checkers operate as black boxes, suffering from internal hallucination and absence of live citations.

TruthLens Solution: An end-to-end, multi-signal verification ecosystem that accepts raw text, PDF documents, Word files, or live URLs, extracts atomic propositions, gathers real-time corroborative web evidence across a 4-tier domain hierarchy, audits hallucination risk via mathematical grounding, distinguishes topical overlap from verbatim plagiarism, calculates a transparent 4-pillar Trust Score (0–100), and exports audit-ready dossiers in JSON, Markdown, and HTML.

Four-Member Engineering & Viva Voce Allocation Matrix

Member / Track	Core Module & Codebase	Key Responsibilities & Viva Focus	Primary Artifacts
Member 1 AI / LLM & Fine-Tuning Specialist	training/train_qwen.py qwen-fact-checker/	Qwen 2.5 3B architecture, 4-bit NF4 quantization, LoRA rank/alpha mechanics, SFTTrainer, memory budget on RTX 5050, loss convergence, PyTorch AMP constraints.	Trained LoRA adapter weights, train.jsonl, PEFT config.
Member 2 Input & Evidence Mining Specialist	input_processor.py app.py (Search Engine)	Multi-format parser (PDF, DOCX, TXT, URL scraping), atomic claim decomposition, DuckDuckGo search orchestration, 4-tier domain credibility scoring, rate limiting.	Document ingestion pipeline, entity extraction, credibility hierarchy.
Member 3 Originality & Similarity Specialist	plagiarism_detector.py similarity_analyzer.py	N-gram shingling & winnowing, Jaccard token overlap, SequenceMatcher Gestalt pattern, TF-IDF cosine similarity, differentiation matrix: Topical Overlap vs. Plagiarism.	Plagiarism engine, topical alignment classifier, HTML word highlighter.
Member 4 Trust Engine & System Lead	trust_engine.py report_generator.py app.py (Gradio UI)	Evidence grounding ratio, 5-tier hallucination risk, 4-pillar Trust Score formula (35/25/25/15), 10-section JSON/MD dossier generator, Gradio 6 reactive UI, 14 automated tests.	Trust engine, report dossier exporter, end-to-end integration test suite.

End-to-End System Execution Flow

Step 1: Ingestion → Clean text extracted from raw paste, PDF stream, Word XML, or URL DOM.
Step 2: Decomposition → Article split into sentences, conjunction-decoupled into atomic factual claims.
Step 3: Web Mining → Multi-query search via DuckDuckGo, gathering top 6 snippets, filtering social/tabloid noise.
Step 4: Source Scoring → URLs matched against Tier 1 (.gov/.edu), Tier 2 (Reuters/BBC), Tier 3 (Wikipedia), or Penalized.
Step 5: LLM Verification → Fine-tuned Qwen 2.5 3B evaluates claim vs evidence: SUPPORTED, REFUTED, or UNCERTAIN.
Step 6: Originality Audit → Evaluates Jaccard, N-grams, Gestalt match, and TF-IDF cosine to isolate true plagiarism from topical words.
Step 7: Hallucination Audit → Matches entities between claims and evidence to yield strict Grounding Score.
Step 8: Trust Calculation → 4-pillar weighted composite score calculated (0–100) with letter grade (A+ to F).
Step 9: Dossier Export → 10-section structured JSON, formatted Markdown, and interactive HTML rendered in Gradio UI.

2. Track 1 (Member 1): Large Language Models, QLoRA Fine-Tuning & Adapter Optimization

Primary Role: Machine Learning Engineer — LLM Architecture, Model Quantization, PEFT Training, GPU Acceleration.

2.1 The Academic Motivation & Problem Statement ('The Why')

Standard pre-trained foundation models (e.g., base LLaMA, standard Qwen) possess broad conversational knowledge but are prone to *syllogistic drift* and conversational agreeableness: when asked to verify a claim, they often validate falsehoods or generate verbose, unparseable disclaimers. Furthermore, full parameter fine-tuning of a 3-billion-parameter model in 16-bit float requires over 24 GB of VRAM—completely unfeasible on consumer or laptop hardware. **QLoRA (Quantized Low-Rank Adaptation)** solves this by freezing the base model in 4-bit precision and training low-rank parameter matrices on targeted attention and MLP projections.

2.2 Deep Architectural & Mathematical Foundations ('The How')

Base Model Choice: Qwen 2.5 3B Instruct
Chosen for state-of-the-art reasoning density per parameter, modern Rotary Position Embeddings (RoPE) supporting long contexts, SwiGLU non-linear activations, and superior multilingual/factual benchmark performance over LLaMA-3.2-3B.

4-Bit NormalFloat (NF4) Quantization Mathematics:
Standard uniform INT4 quantization introduces high rounding errors on normally distributed weights. NF4 constructs an information-theoretically optimal quantile grid where each bin has an equal number of expected values under a zero-mean unit-variance normal distribution $N(0, 1)$.
Double Quantization (DQ): Quantizes the quantization constants (scales) themselves from FP32 to FP8, saving an additional 0.37 bits per parameter (~300MB VRAM savings on 3B models).

LoRA Mathematical Formulation:
Given a frozen weight matrix $W_0 \in \mathbb{R}^{d \times k}$, the updated weight during forward pass is:
$W' = W_0 + \Delta W = W_0 + \frac{\alpha}{r} (B \cdot A)$
where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank decomposition matrices, initialized with $A \sim N(0, \sigma^2)$ and $B = 0$ (so $\Delta W = 0$ at start).
• **Rank ($r = 16$):** Dimension of the low-rank subspace.
• **Alpha ($\alpha = 32$):** Scaling hyperparameter ($ \frac{\alpha}{r} = 2.0$), stabilizing gradient magnitudes.
• **Target Projections:** All 7 linear projection layers: q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj.

VRAM Memory Budget Breakdown (NVIDIA RTX 5050 Laptop GPU — 7.93 GB Total)

Memory Component	Full FP16 Training	TruthLens 4-Bit QLoRA	Engineering Rationale
Base Model Weights	6.2 GB (16-bit)	~1.9 GB (4-bit NF4)	Compressed via BitsAndBytesConfig with Double Quant.
Trainable Adapters	N/A	~60 MB (FP16)	Only A and B matrices are trainable (r=16).
Optimizer States	12.4 GB (AdamW FP32)	~0.15 GB (Paged AdamW 8-bit)	Paged memory paging to CPU RAM if spikes occur.
Activation Memory	4.5 GB	~1.2 GB	Gradient Checkpointing enabled (recomputes forward).
Total VRAM Usage	**> 23 GB (OOM Crash)**	**~3.4 GB (Comfortable fit)**	Leaves 4.5 GB headroom for OS and display.

2.3 Minute Implementation & Training Metrics ('The Code & Numbers')

Training Script: training/train_qwen.py using HuggingFace trl.SFTTrainer and peft.LoraConfig.
Hyperparameters:
• num_train_epochs = 5 | per_device_train_batch_size = 1 | gradient_accumulation_steps = 4 (Effective batch size = 4)
• learning_rate = 2e-4 with linear warmup and cosine decay.
• optim = 'paged_adamw_8bit' | gradient_checkpointing = True | max_grad_norm = 0.0 (disabled).
• **Critical Windows AMP Workaround:** Set fp16=False and bf16=False in TrainingArguments. Under Windows with CUDA 13.2 and current PyTorch/Accelerate, PyTorch's native BF16 automatic mixed precision scaler triggers an internal gradient scaler assertion error. Disabling AMP forces pure FP32 accumulation through BitsAndBytes 8-bit optimizer, running completely error-free.
Empirical Training Results (Completed Run):
• Total Steps: **100 steps** across 80 training examples (5 epochs).
• Wall-clock Training Runtime: **312.2 seconds** (5.2 minutes) at 0.32 steps/sec.
• Training Loss Trajectory: Commenced at **~1.52** → Dropped to **0.4798** final loss.
• Mean Token Accuracy: Reached **97.95%** on factual verdict tokens.
• Saved Artifact: Adapter files saved to qwen-fact-checker/ (adapter_config.json, adapter_model.safetensors).

2.4 Member 1 Viva Voce Defense Sheet (Top Examiner Questions)

Q1: Why did you choose QLoRA instead of standard LoRA or full fine-tuning?

Full fine-tuning requires updating 3B FP16 parameters plus storing 32-bit optimizer states (16 bytes/param for AdamW = ~48GB memory). Standard LoRA still loads the base model in FP16 (6GB VRAM), leaving minimal headroom for KV caches. QLoRA quantizes the base model into 4-bit NF4, slashing base memory to ~1.9GB. We only train 16-rank adapter matrices (60MB), enabling full training on an 8GB laptop GPU.

Q2: What is NF4 quantization, and why is it superior to FP4 or INT4?

NormalFloat4 is an information-theoretically optimal quantile quantization scheme for normally distributed data. Foundation model weights follow Gaussian distributions centered at zero. Uniform INT4 spaces quantization levels equally, wasting precision in the tails. NF4 spaces quantiles such that each 4-bit bin contains equal probability mass, preserving empirical perplexity almost identically to 16-bit weights.

Q3: Why did you target all projection layers (q, k, v, o, gate, up, down) rather than just q and v?

Original LoRA literature (Hu et al.) only adapted q_proj and v_proj for parameter efficiency. However, recent empirical work (Dettmers et al., QLoRA paper) proves that adapting both attention projections (q, k, v, o) and feed-forward MLP projections (gate, up, down) captures factual transformation knowledge much better and recovers 99% of full fine-tuning performance.

Q4: Why did you disable mixed precision (fp16/bf16=False) on Windows?

On Windows with CUDA 13.2, Accelerate's BF16 automatic mixed precision scaler triggers a known PyTorch GradScaler assertion bug. By disabling native AMP and relying on BitsAndBytes paged_adamw_8bit optimizer, weights compute in 16-bit while optimizer states remain 8-bit, running stably without accuracy degradation.

Q5: How does the model output verdicts during inference?

We format input prompts using Qwen's ChatML template: '<|im_start|>system\nVerify factual claims...<|im_end|>\n<|im_end|>'. The fine-tuned model deterministically emits normalized labels: 'SUPPORTED', 'REFUTED', or 'PARTIALLY_SUPPORTED' accompanied by brief justification text.

3. Track 2 (Member 2): Multi-Modal Ingestion, Search Orchestration & Credibility Tiering

Primary Role: Data & Search Infrastructure Engineer — Multi-Format Parsing, DOM Extraction, Evidence Mining, Domain Tiering.

3.1 The Academic Motivation & Problem Statement ('The Why')

A verification system is only as good as the evidence it retrieves. In real-world enterprise environments, documents do not arrive as pristine text strings: they arrive as scanned PDFs with broken line wraps, Word documents with complex XML hierarchies, or cluttered web pages polluted with ads and navbars. Furthermore, performing naive web searches often surfaces low-quality blog posts, Reddit opinions, or circular citations that pollute the verification pipeline. Member 2 owns the ingestion reliability and authoritative evidence mining.

3.2 Ingestion Architecture Across 5 Data Modalities ('The How')

Module: input_processor.py with unified dispatcher ingest_content(source_type, content, file_obj).

1. PDF Processor (pypdf.PdfReader):

• Iterates across all pages, extracting raw text streams.

• *De-hyphenation Engine:* Regex r'(\w+)-\n(\w+)' → r'\1\2' repairs hyphenated words split across line breaks (e.g., 'trans-\nformer' → 'transformer').

• Paragraph grouping: Collapses consecutive single newlines into spaces while preserving double newlines as paragraph boundaries.

2. Word Document Processor (docx.Document):

• Traverses document XML paragraphs, extracting text blocks and ignoring invisible formatting run metadata.

• Legacy .doc binary handling: Detects OLE compound document headers and provides structured fallback messages.

3. Plain Text / Markdown: Decodes UTF-8 with automatic fallback to Latin-1/CP1252 for legacy Windows encodings.

4. Live URL Web Scraper:

• HTTP request with customized browser User-Agent headers to circumvent basic anti-scraping blocks.

• BeautifulSoup4 cleans boilerplates: completely strips <script>, <style>, <nav>, <footer>, <header>, <aside>, <form>.

• Semantic tag hierarchy: Prioritizes <article>, then <main>, then body paragraph collections <p> to extract pure editorial content.

3.3 Atomic Claim Decomposition Algorithm

Compound sentences in journalism frequently contain multiple independent factual propositions (e.g., *'The telescope launched in 2021, but its primary mirror failed to deploy.'*). Verifying compound sentences as a single unit leads to catastrophic false negatives.

Algorithm (extract_atomic_claims in app.py):

1. Tokenizes article into sentences via regex terminal boundary matching (r'(?<=[.!?])\s+').
2. Filters trivial phrases (< 5 words).
3. Decouples adversative and coordinating conjunctions using lookahead regex: r',\s+(?=(?:but|while|whereas|although|though|and)\b)'.
4. Splits appositive clauses (r',\s+(?=which\b)') and parses compound coordinate clauses.
5. Strips leading conjunctions and normalizes terminal punctuation, producing atomic, independent verification targets.

3.4 4-Tier Source Credibility & Authority Model

TruthLens implements a deterministic domain authority hierarchy (get_source_credibility and compute_authority_score in app.py). Evidence sources are scored between 0.0 and 1.0 and categorized into tiers:

Tier Level	Domain Criteria & Exemplars	Score / Weight	Pipeline Treatment
Tier 1 Government & Science	.gov, .edu, .mil, .ac.uk nasa.gov, who.int, nature.com, nih.gov, cdc.gov, science.org	**0.94 – 0.95** Authority: +35	Primary authoritative ground truth. Highest verification weight in LLM prompt.
Tier 2 Major News & Fact-Check	reuters.com, apnews.com, snopes.com, factcheck.org, bbc.com, nytimes.com	**0.85** Authority: +26	Reputable investigative reporting; trusted for contemporary events.

Tier 3 Specialist Portals & Wiki	wikipedia.org, sciencedirect.com, mayoclinic.org, livescience.com	**0.68 – 0.72** Authority: +18–22	Domain reference; max 1 Wikipedia source allowed per claim to avoid bias.
Penalized / Blacklist Social / Opinion / Tabloid	reddit.com, twitter.com, x.com, quora.com, tiktok.com, dictionary.com	**0.15** Penalty: -20 to -35	Filtered or severely down-weighted; prevented from verifying claims.

3.5 Member 2 Viva Voce Defense Sheet (Top Examiner Questions)

Q1: How do you extract clean text from a PDF with complex multi-column layouts?

We utilize pypdf stream extraction paired with our custom clean_extracted_text post-processor. It uses regular expression lookarounds to repair soft-hyphen line splits ('word-\npart' -> 'wordpart') and normalizes whitespace while reconstructing paragraph semantic boundaries based on double newline delimiters.

Q2: Why do you decompose sentences into 'atomic claims' before verification?

Journalistic writing combines multiple factual statements into single sentences with clauses ('X occurred, but Y did not'). If passed whole to an LLM, the model either gives an ambiguous verdict or hallucinates. Decomposing into atomic propositions allows independent evidence retrieval and independent verification for every factual assertion.

Q3: How does your search engine avoid hitting DuckDuckGo rate limits?

We implemented three defensive mechanisms in app.py: (1) SEARCH_RETRIES = 2 with exponential backoff; (2) SEARCH_RETRY_DELAY = 1.5s between retry attempts; and (3) SEARCH_REQUEST_DELAY = 0.35s between individual claim searches to avoid rapid socket churn. We also enforce MAX_SEARCH_RESULTS = 6 to bound payload sizes.

Q4: Why is Wikipedia limited to at most 1 source per claim?

While Wikipedia is valuable for general knowledge (Tier 3, 0.68 score), it is crowd-sourced and can be vandalized or mirror the very claims being checked. To enforce genuine evidence diversity, app.py sets MAX_WIKIPEDIA_SOURCES = 1 and MAX_SAME_DOMAIN_SOURCES = 1, ensuring multi-domain consensus.

Q5: What happens if an input document is an unsupported or corrupted file?

The input_processor.py module implements defensive try/except blocks around each extractor. If a file is unreadable, corrupted, or unsupported, it returns a structured dictionary with success=False, word_count=0, and a user-friendly error message that Gradio renders gracefully without crashing the server.

4. Track 3 (Member 3): Originality Verification, Plagiarism Detection & Topical Similarity Engine

Primary Role: NLP Algorithm Engineer — Lexical Fingerprinting, Syntactic Matching, Semantic Alignment, Plagiarism Differentiation.

4.1 The Academic Motivation & Problem Statement ('The Why')

Commercial plagiarism detection tools (such as Turnitin, Copyscape, or Grammarly) suffer from a notorious flaw: **the Domain Terminology False Positive**. In technical, scientific, or political articles, independent authors writing about the same event inevitably use the same core terms (e.g., *'exoplanet atmosphere', 'James Webb Space Telescope', 'transmission spectroscopy'*). Turnitin flags this as 30% plagiarism! TruthLens solves this fundamental problem through a dual-engine architecture that mathematically distinguishes **unethical verbatim copying / paraphrasing** from **legitimate topical and thematic overlap**.

4.2 Dual-Engine Algorithmic Architecture ('The How')

Engine 1: Hybrid Plagiarism Detector (plagiarism_detector.py)
Designed to detect intellectual property theft and unauthorized verbatim or structural duplication:
• **Word N-Gram Shingling ($n=4$):** Converts texts into sliding windows of 4 consecutive word tokens (shingles). Computes shingle containment:
$ ext{Containment}(S, R) = rac{| ext{Shingles}(S) \cap ext{Shingles}(R)|}{| ext{Shingles}(S)|}$
• **Jaccard Lexical Similarity:** Measures word set intersection over union after stopword stripping:
$J(A, B) = rac{|A \cap B|}{|A \cup B|}$
• **Gestalt Pattern Matching (difflib.SequenceMatcher):** Identifies longest contiguous matching substring blocks. If a continuous matching block ≥ 6 words is detected, it is flagged as verbatim copy.
• **Visual HTML Mark Highlighter:** Automatically wraps matching contiguous blocks of 3+ words in <mark class='plag-highlight'> tags for visual evidence display in the UI.

Engine 2: Topical & Semantic Similarity Analyzer (similarity_analyzer.py)
Designed to measure conceptual alignment across literature without penalizing originality:
• **Lightweight Morphological Stemmer:** Deterministic suffix stripper for English terms (ing, ies, es, ed, ly, s) that unifies morphological variants without heavy NLTK overhead.
• **L2-Normalized Term Frequency (TF) Vector Space:** Maps token stems to term frequency weights normalized by Euclidean norm: $ ec{v}_w = rac{ ext{count}(w)/N}{\| ec{v}\|_2}$.
• **Cosine Similarity:** Measures topical orientation angle: $\cos(ec{u}, ec{v}) = \sum_{w \in U \cap V} u_w \cdot v_w$.

4.3 The Similarity vs. Plagiarism Categorical Decision Matrix

The core innovation of Phase 2 is the multi-threshold decision matrix (analyze_pair in similarity_analyzer.py) that assigns texts to one of five mutually exclusive relationship categories:

Category	Mathematical Trigger Condition	Is Plagiarism?	Academic Meaning & Interpretation
VERBATIM COPY	Sequence ratio ≥ 0.70 OR Continuous block $\ge \max(6, 0.6 imes \|words\|)$	**TRUE (High Risk)**	Direct copy-paste of sentence structures without quotes or attribution. Severe academic integrity violation.
HEAVY PARAPHRASE	Composite Sim $\ge 65\%$ AND Jaccard Index ≥ 0.50	**TRUE (High Risk)**	Sentence structure copied with synonym substitution or rearranged clauses without original intellectual contribution.
TOPICAL OVERLAP	Topical Sim $\ge 25\%$ OR Composite $\ge 25\%$ OR ≥ 2 shared terms	**FALSE (Safe)**	Both texts discuss the identical real-world event/subject, sharing domain terminology, but phrasing is completely independent.
SHARED TERMINOLOGY	Composite Sim $\ge 12\%$ OR ≥ 1 shared domain keyword	**FALSE (Safe)**	Incidental overlap on standard technical vocabulary (e.g., 'emissions', 'greenhouse'). Completely normal.
ORIGINAL EXPRESSION	Composite Sim < 12% and no shared key terms	**FALSE (Safe)**	Fully independent writing with distinct vocabulary and phrasing.

4.4 Member 3 Viva Voce Defense Sheet (Top Examiner Questions)

Q1: What is the mathematical difference between Jaccard similarity and Cosine similarity?

Jaccard similarity measures set overlap: $|A \cap B| / |A \cup B|$. It is binary (a word is either present or absent) and treats all tokens equally. Cosine similarity evaluates the angle between term-frequency vectors in an inner product space. It accounts for term frequency weighting and vector magnitude normalization, making it much better for measuring topical orientation rather than exact phrasing.

Q2: Why do traditional plagiarism checkers fail on scientific papers?

Traditional checkers rely strictly on string matching and n-gram overlap without domain semantics. In specialized topics, authors must use domain-specific terms ('photosynthesis', 'chloroplast', 'ATP synthase'). Traditional checkers mark these shared terms as plagiarized text. TruthLens uses our Categorical Matrix: if shared words are present but continuous sequence length is < 6 words and Jaccard < 0.50, it is classified as legitimate Topical Overlap.

Q3: What is n-gram shingling, and why did you pick n=4?

An n-gram shingle is a contiguous sequence of n tokens. Single words (n=1) produce massive false positives due to common vocabulary. Large n (n=8+) fails to catch paraphrasing where clauses are reordered. In NLP literature (Broder et al., Winnowing), 4-grams strike the optimal empirical balance: an identical 4-word sequence has low random probability in English and strongly indicates copied phrasing.

Q4: How does difflib.SequenceMatcher determine the longest common match?

SequenceMatcher uses the Gestalt pattern matching algorithm (Ratcliff and Obershelp). It recursively finds the longest contiguous common matching subsequence between two token lists, then matches remaining left and right pieces. It operates in O(N^2) worst case but O(N) heuristic time on typical text, returning the exact length of uninterrupted verbatim strings.

Q5: How does the system highlight plagiarized segments in the web interface?

The highlight_matched_segments function in plagiarism_detector.py compares suspect and source token lists via SequenceMatcher. When a matching block with size >= 2 is found, the corresponding indices in the suspect sentence are wrapped with ... tags, which CSS renders in transparent amber highlighting.

5. Track 4 (Member 4): Evidence Grounding, Hallucination Auditing, Transparent Trust Scoring & Dossier System

Primary Role: Systems Architect & Lead — Trust Mathematics, Hallucination Detection, Dossier Generator, System Integration, Testing.

5.1 The Academic Motivation & Problem Statement ('The Why')

Modern AI systems are plagued by the 'black-box problem.' Many AI verification prototypes output arbitrary numbers like '84% True' without mathematical justification. If a user asks *'Why is this score 84%?'*, the system cannot explain itself. Furthermore, AI hallucination is multifaceted: a claim may be contradicted by evidence, or it may simply be uncorroborated because evidence does not exist. Member 4 owns the **strict evidence grounding metric**, the **transparent 4-pillar Trust Score formula**, the **report dossier generator**, and overall **system integration**.

5.2 Evidence-Based Hallucination Detector ('The How')

Module: trust_engine.py → HallucinationDetector.
Unlike naive approaches that merely ask an LLM *'Did you hallucinate?'*, TruthLens computes hallucination risk mathematically based on **factual entity grounding**:
1. **Factual Entity Extraction (extract_factual_entities):** Uses regex to extract (a) multi-word capitalized proper nouns (e.g., 'James Webb', 'Albert Einstein'), (b) numeric values and dates (e.g., '2021', '3.8'), and (c) non-stopword content words > 2 characters.
2. **Grounding Coverage Ratio:** Measures what proportion of the claim's factual entities appear in the retrieved evidence snippets:
$ ext{Grounding Ratio} = rac{| ext{Entities}_{ ext{claim}} \cap ext{Tokens}_{ ext{evidence}}|}{| ext{Entities}_{ ext{claim}}|}$
3. **5-Tier Hallucination Risk Classification:**
• **CONTRADICTED:** Verdict is REFUTED. Factual mismatch with credible evidence. **Risk: 92.0%**.
• **UNVERIFIABLE:** No evidence retrieved OR Grounding < 20% with UNCERTAIN verdict. **Risk: 65.0%**.
• **UNSUPPORTED:** Sources discuss topic, but specific assertion is missing (Grounding < 50%). **Risk: 55.0%**.
• **PARTIALLY GROUNDED:** Partial evidence corroboration. **Risk: 35.0%**.
• **FAITHFUL:** Verdict is SUPPORTED and Grounding ≥ 70%. **Risk: 4.0% – 15.0%**.
Article Grounding Score: $ ext{Grounding Score} = 100.0 - ext{Mean Hallucination Risk}$.

5.3 The Transparent Multi-Pillar Trust Score Formula

Module: trust_engine.py → TrustScoreEngine.calculate_trust_score.
The overall Trust Score (0–100) is calculated across 4 transparent, auditable pillars with fixed weights:

```
Trust Score = (0.35 * Fact_Verification) + (0.25 * Source_Credibility) +
              (0.25 * Evidence_Grounding) + (0.15 * Content_Originality)
```

Detailed Pillar Formulations:
• **Pillar 1: Fact Verification Integrity (35% Weight):**
$ ext{Raw Fact Score} = rac{(N_{ ext{supp}} imes 100) + (N_{ ext{part}} imes 60) + (N_{ ext{uncert}} imes 35) + (N_{ ext{ref}} imes 0)}{N_{ ext{total}}}$
Refuted Assertion Penalty: If $N_{ ext{ref}} > 0$, apply penalty: $ ext{Penalty} = \min(25.0, N_{ ext{ref}} imes 12.0)$ to prevent false balance.
• **Pillar 2: Source Credibility & Authority (25% Weight):**
Mean credibility of cited sources: $ ext{Credibility Score} = rac{1}{|Sources|} \sum s_i imes 100.0$. Default = 35.0 if no sources.
• **Pillar 3: Evidence Grounding / Hallucination Resistance (25% Weight):**
Directly equals the article's Grounding Score ($100.0 - ext{Overall Hallucination Risk}$).
• **Pillar 4: Content Originality (15% Weight):**
Directly equals $100.0 - ext{Plagiarism Index}$ from the originality engine.

Executive Trust Grade Rubric

Score Range	Tier Grade	Rating Label	Executive Editorial Interpretation
82.0 – 100.0	**A+**	VERIFIED HIGH TRUST	Article is solidly grounded in authoritative sources with verified facts and original writing.
68.0 – 81.9	**A**	GENERALLY RELIABLE	Majority of claims verified by credible sources; minor unverified or paraphrased passages.
50.0 – 67.9	**B**	MODERATE / MIXED TRUST	Contains mixed verification results, uncorroborated assertions, or moderate similarity.
32.0 – 49.9	**C**	LOW TRUST / CAUTION	Multiple unverified assertions, questionable source backing, or noticeable hallucination risk.
0.0 – 31.9	**F**	HIGHLY UNRELIABLE	Substantial contradicted claims, fabricated assertions, or excessive plagiarism detected.

5.4 Verification Report Generator (Phase 4 Dossier System)

Module: report_generator.py.
Generates audit dossiers conforming to a standardized 10-section JSON schema:
[1] metadata | [2] article_summary | [3] trust_score | [4] claims_verified | [5] evidence_sources | [6] hallucination_audit | [7] originality_audit | [8] similarity_analysis | [9] key_recommendations | [10] appendix.
Multi-Format Exporters:
• **JSON Exporter (generate_json_report):** Full structured machine-readable export.
• **Markdown Exporter (generate_markdown_report):** Audit-ready document with formatted summary tables, claim verdicts, risk badges, and bibliography.
• **HTML Data URI Exporter:** Uses base64.b64encode and urllib.parse.quote to provide instant client-side file downloads directly inside Gradio without requiring local server storage.

5.5 Member 4 Viva Voce Defense Sheet (Top Examiner Questions)

Q1: Why did you choose the specific weights (35%, 25%, 25%, 15%) for the Trust Score?

The weights reflect epistemic importance in factual verification: Fact Verification (35%) is paramount because a single refuted fact can invalidate an article. Evidence Authority (25%) and Hallucination Resistance (25%) ensure that claims are corroborated by peer-reviewed or authoritative institutions rather than rumors. Originality (15%) guards against unattributed copying while avoiding penalizing domain terminology.

Q2: Why do you apply a penalty when refuted claims are present?

Without a penalty, an article with 5 true claims and 1 false claim would receive an 83% trust score ('A' grade). In journalism and medicine, a single false claim (e.g., 'Vaccine contains microchips') can be lethal. Our formula subtracts min(25.0, refuted_count * 12.0), immediately downgrading such articles to 'B' or 'C' grades.

Q3: How does your hallucination detector avoid simply trusting the LLM's own self-evaluation?

Self-evaluation by LLMs is unreliable because hallucinating models also hallucinate that they are correct. TruthLens uses deterministic Entity Grounding: we parse named entities, dates, and numbers from the claim via regular expressions, then verify their presence in the retrieved web text. The grounding ratio is computed mathematically before the verdict is assigned.

Q4: How do client-side downloads work in Gradio without filling the server hard drive?

In report_generator.py, we encode the generated Markdown and JSON dossiers into UTF-8 Base64 strings and format them as HTML data URIs: href='data:application/json;base64,...' download='report.json'. When the user clicks the download button in the browser, their browser handles the file save locally without writing temporary files to the server disk.

Q5: What automated tests verify your pipeline, and how do they prevent regressions?

We maintain 14 automated test scripts in the tests/ directory. These include unit tests for report generation, trust scoring, and document ingestion, plus test_phase4_pipeline.py which executes the full end-to-end pipeline across all 6 engines. All 14 tests run with 100% pass rates.

6. Master Viva Voce Defense: Cross-Cutting Architectural Questions

Questions Examiners Ask to Test Team Synergy and System Mastery

In capstone defenses, external examiners frequently ask overarching architectural and comparative questions to test whether all four group members understand how their respective components interface with each other. Here are the top comprehensive answers:

Q1: How does the system handle conflicting evidence (e.g., Source A says True, Source B says False)?

When retrieved evidence sources conflict, TruthLens resolves the discrepancy through a two-stage arbitration process. First, Member 2's Credibility Engine scores the conflicting sources based on domain tiering (a Tier 1 source like NASA or Nature carries a weight of 0.95, overriding a Tier 3 blog with 0.45). Second, both evidence snippets are formatted into Qwen 2.5's prompt with their authority scores. If evidence remains in genuine parity with no consensus, the fine-tuned model emits 'UNCERTAIN', and Member 4's trust engine marks the claim as 'UNVERIFIABLE' rather than hallucinating a false certainty.

Q2: Why did you build a local fine-tuned QLoRA model rather than calling the OpenAI GPT-4 API?

We deliberately engineered a local fine-tuned model for three compelling technical reasons: (1) Academic & Architectural Ownership: Fine-tuning Qwen 2.5 with 4-bit QLoRA demonstrates mastery over quantization, PEFT, loss optimization, and GPU memory physics. (2) Privacy & Zero Data Exfiltration: Enterprise document verification often involves proprietary or confidential drafts that cannot be legally transmitted to third-party cloud APIs. (3) Deterministic Latency & Zero API Cost: Local inference on the RTX 5050 GPU runs indefinitely at zero marginal financial cost and immunity to external API rate limits or deprecations.

Q3: What are the primary computational bottlenecks during an end-to-end verification run?

Profiling the pipeline reveals that network I/O during multi-query web search constitutes ~65% of elapsed time (due to latency in scraping search result pages). Model inference on the 4-bit quantized Qwen 2.5 3B takes ~25% of runtime (~1.2 seconds per claim on the RTX 5050). The mathematical engines (Jaccard, SequenceMatcher, TF-IDF cosine, Entity Grounding, Trust Score computation) are extremely lightweight, consuming < 10% of total runtime combined.

Q4: How did you clean and organize the project folder, and why was that necessary?

Initially, the workspace contained 28 cluttered files with scattered test scripts, legacy command-line tools, and unorganized datasets. We restructured the workspace into 6 clean root production engines (app.py, input_processor.py, similarity_analyzer.py, trust_engine.py, report_generator.py, plagiarism_detector.py) and 4 organized subdirectories (tests/ with 14 automated tests, training/ with train_qwen.py and datasets, qwen-fact-checker/ with trained weights, and legacy/ with deprecated CLI scripts). We also created README.md explaining every file in plain English.

Q5: Where would you take this project next if extended for an M.Tech thesis or commercial startup?

Key future enhancements include: (1) Graph-RAG Integration: Building a knowledge graph of extracted claims to detect temporal contradictions across long-term news cycles; (2) Multi-Lingual Claim Verification: Expanding training data to verify regional language journalism; (3) Multi-Modal OCR: Adding layout-aware OCR (e.g., Donut or Nougat) for scanned physical newspapers and infographics; and (4) Blockchain Provenance: Writing the generated JSON dossier hashes to an immutable ledger for verifiable editorial auditing.

7. Complete Project File Inventory & Technical Roles

The following table provides a complete reference for every file currently active in the project root and subdirectories:

File / Directory	Lines / Size	Primary Technical Function	Lead Member
app.py	~5,700 lines	Main Gradio 6 web application, pipeline orchestration, claim extraction, search layer.	Member 4 / All
input_processor.py	628 lines	Multi-modal document parser (PDF, Word, TXT, live URLs with BeautifulSoup4).	Member 2
similarity_analyzer.py	334 lines	Topical similarity engine, TF-IDF cosine alignment, 5-category differentiation matrix.	Member 3
trust_engine.py	401 lines	Entity grounding calculator, 5-tier hallucination auditor, 4-pillar Trust Score engine.	Member 4
report_generator.py	600 lines	10-section JSON schema builder, Markdown dossier exporter, HTML view renderer.	Member 4

plagiarism_detector.py	660 lines	Hybrid plagiarism engine, 4-gram shingling, Jaccard similarity, HTML mark highlighter.	Member 3
training/train_qwen.py	329 lines	QLoRA 4-bit fine-tuning script using SFTTrainer, BitsAndBytes, and PEFT.	Member 1
qwen-fact-checker/	Directory (~60MB)	Trained LoRA adapter weights (adapter_model.safetensors, adapter_config.json).	Member 1
tests/	14 Test Scripts	Automated unit and integration test suite (Phase 1–4, 100% pass rate).	All Members
README.md	122 lines	Plain-English, jargon-free guide on what every file does and how to run the app.	All Members

[VIVA VOCE DEFENSE STRATEGY]

Summary for Viva Voce Presentation: Present the project with confidence! Emphasize the live multi-tier search, the local 4-bit QLoRA adaptation on consumer GPU, the mathematical differentiation of topical overlap versus plagiarism, and the 100% transparent 4-pillar Trust Score formula. Every component is backed by working code and passing tests.